

School of Informatics

Informatics Project Proposal

Multi-Tenant Scheduling Architecture for Latency-Critical AI Inference on Shared RAN Infrastructure

UID: [Insert Student UID]

Date: April 2026

Tutor: [Insert Tutor Name]

Supervisor: [Insert Supervisor Name]

I give permission for future Informatics Project Proposal classes to read an anonymous version of this document.

This page does not count towards the page limit.

Abstract

The evolution toward 6G mobile networks has catalyzed the AI-RAN paradigm, envisioning a profound convergence of Radio Access Network (RAN) infrastructure and artificial intelligence workloads. Modern GPU-accelerated base stations frequently exhibit significant, temporally bursty spare compute capacity. The central challenge addressed in this research is how to safely harness this underutilized edge compute for latency-critical AI inference tasks without compromising the stringent Service Level Agreements (SLAs) of primary telecommunication functions. This project proposes the design and evaluation of a cloud-native, multi-tenant scheduling architecture. Adopting a strict “RAN-first” operational philosophy, it utilizes advanced workload isolation to dynamically multiplex containerized AI inference into the distributed unit (DU), guaranteeing zero degradation to network reliability.

Declaration of Own Work

I hereby declare that the work presented in this document is my own. The use of Generative AI to assist in the formatting and structuring of this proposal has been fully acknowledged.

1. Motivation

1.1. Problem Statement

As telecommunications networks move toward 6G, recent AI-RAN work increasingly frames the RAN as more than a communications substrate, namely as a cloud-native, distributed platform that can host both primary Radio Access Network (RAN) functions and selected AI workloads on shared infrastructure [1, 2, 3]. A compelling use case within this convergence is latency-sensitive edge inference, where placing model execution close to the radio path can reduce transport delay and keep demanding AI workloads off user devices [1, 4, 5].

RAN infrastructure is also provisioned for peak demand, which leaves meaningful spare capacity outside worst-case conditions [1, 2]. Recent profiling from *Weaver* makes this point concrete: even in a pessimistic four-DU peak-load setting, about 40% spare compute remains, and production-trace replays show that spare SM capacity can still be around 85% during peak daytime periods across six cell sites from three operators [6]. The same study shows that this headroom is bursty within a site but complementary across sites, which makes it valuable yet difficult to harvest with static sharing policies [6].

The central challenge, then, is safe coexistence between workloads with very different timing contracts. DU processing follows tight slot-level deadlines, while edge inference must meet service latency targets without degrading the primary radio function [1, 6, 4]. Existing GPU-sharing mechanisms expose the trade-off: hardware partitioning such as NVIDIA’s Multi-Instance GPU (MIG) offers strong isolation and is therefore a useful hard-isolation option or comparison baseline, but static partitions are a poor fit for bursty millisecond-scale spare cycles; more elastic sharing can expose more headroom, but makes deadline protection harder to guarantee [6, 5]. This creates a clear need for a scheduling architecture that keeps a strict “RAN-first” policy while still exposing usable spare compute to multi-tenant edge AI services.

1.2. Research Hypothesis and Objectives

1.2.1. Research Hypothesis

Recent profiling of accelerated RAN infrastructure, especially in *Weaver*, shows why AI *training* is a comparatively favorable co-located workload: LDPC-heavy RAN processing is bandwidth-dominated, while foundation-model training is more compute-dominated, giving the two workloads a degree of resource complementarity [6]. *Weaver* also shows that stabilizing bursty spare compute can raise training throughput from about 54 TFLOPs to about 142 TFLOPs under similar average SM availability, highlighting how strongly workload performance depends on the temporal shape of spare capacity [6]. This contrast makes inference the more demanding next step for AI-and-RAN scheduling.

Deploying AI *inference* is harder because inference is not a single resource profile. Transport, model execution, and response delivery stress different parts of the system, and the most latency-sensitive phases leave much less room for contention with primary RAN functions [4, 6, 5]. Furthermore, enabling this as an edge service requires multi-tenancy. *Paradrop* showed that lightweight, containerized services can be placed at the extreme edge, but it did not need the sub-millisecond priority handling expected in a DU-adjacent RAN environment [7].

Therefore, the core **Research Hypothesis** of this proposal is: *By employing a hierarchical scheduling architecture that combines a slow-timescale, cloud-native orchestration layer for multi-tenant isolation with a fast-timescale, “RAN-first” local MAC scheduler, it is possible to dynamically multiplex latency-critical AI inference workloads onto the spare compute capacity of*

GPU-accelerated RAN Distributed Units (DUs). We hypothesize that this architecture can satisfy the real-time latency bounds of edge inference while guaranteeing zero Service Level Agreement (SLA) violations for primary telecommunications workloads.

1.2.2. Objectives

To test this hypothesis and realize the proposed architecture, this research will pursue the following specific objectives, categorized by architectural layer:

1. **Design a Cloud-Native, Multi-Tenant Orchestration Layer (Macro-level):** Extend extreme-edge orchestration principles, such as those explored in *Paradrop*, to the 6G RAN environment. This involves designing a slow-timescale (seconds to minutes) management plane responsible for tenant onboarding, containerized workload isolation, and SLA-class assignment across the O-Cloud [7, 1, 3].
2. **Develop a “RAN-first” Micro-Scheduler (Micro-level):** Design a fast-timescale (sub-10 millisecond) scheduling algorithm residing at the DU. This scheduler will dynamically allocate bursty spare GPU cycles to inference tasks and compare hard-isolation options such as MIG against more elastic sharing mechanisms to determine which choices best protect baseband deadlines while still exposing useful spare capacity [6, 5].
3. **Implement End-to-End Latency Coordination:** Adapt the end-to-end budgeting logic exemplified by *CORA* so that overall service delay is apportioned across uplink transport, edge execution, and downlink delivery, with the compute budget refined further when different inference phases exhibit different runtime behavior [4].
4. **System Evaluation and Profiling:** Evaluate the proposed hierarchical architecture within a simulated or physical AI-RAN testbed. The primary metrics for success will be demonstrating 0% SLA violations for the primary RAN workload under heavy load, alongside acceptable inference latency and increased overall GPU utilization.

1.3. Timeliness, Novelty, and Significance

1.3.1. Timeliness

The timing is favorable because AI-RAN is moving from broad vision to concrete, cloud-native architectural proposals and deployment workflows [1, 2, 3]. At the same time, large-scale RAN experimentation and latency-critical cellular inference are becoming more concrete through public-cloud emulation platforms such as *CHRONOS* and end-to-end coordination frameworks such as *CORA* [8, 4]. Recent device-RAN-cloud inference work further suggests that SLA-aware placement is becoming a practical systems question rather than a purely conceptual one [5].

1.3.2. Novelty and Relation to Prior Work

Prior work addresses important pieces of this problem, but not the full combination required here. *Beyond Connectivity* and related AI-RAN architecture efforts describe the orchestration substrate for shared AI and RAN infrastructure [1, 2, 3]. *Paradrop* demonstrates lightweight multi-tenancy at the extreme edge [7]. *CORA* coordinates uplink, compute, and downlink for latency-critical inference [4]. *Weaver* shows how to expose RAN-safe spare GPU capacity, but its target workload is foundation model training rather than latency-critical inference [6]. *SLA-Aware Distributed LLM Inference Across Device-RAN-Cloud* further suggests that strict SLAs

may require hard isolation or carefully selected model variants, which makes MIG a useful comparison point rather than a universal answer [5].

The novelty of this research lies in combining these strands into one scheduling problem: determining which inference workloads can share DU-adjacent compute, under what isolation contract, and with what end-to-end latency budget. The project therefore focuses on phase-aware inference profiling, DU-local coexistence control, and multi-tenant orchestration that joins *Weaver*-style spare-compute harvesting with *CORA*-style latency coordination and *Paradrop*-style lifecycle management [6, 4, 7].

1.3.3. Significance

If successful, this research could clarify whether multi-tenant edge inference can be executed natively on RAN infrastructure without degrading network reliability. For network operators, that would provide a blueprint for monetizing otherwise idle GPU cycles without deploying new hardware. For end-users and device manufacturers, it would strengthen the case for low-latency edge support for lightweight, battery-efficient mobile devices. More broadly, the project would contribute a technical foundation for treating AI services as a native component of 6G networks.

1.4. Feasibility

1.5. Beneficiaries

2. Background and Related Work

This section provides the foundational context necessary to understand the proposed scheduling architecture. It first outlines the evolution of cloud-native telecommunications and edge AI, before critically reviewing the existing literature on GPU sharing, edge multi-tenancy, and RAN-aware scheduling to highlight the specific gaps this research will address.

2.1. Background

2.1.1. Cloud-Native O-RAN Architecture

Historically, cellular network deployments relied on monolithic, proprietary hardware appliances, which reinforced vendor lock-in and rigid operational models. Open RAN (O-RAN) marks a shift toward disaggregated baseband functions and standardized open interfaces [3, 1, 2]. A central part of this design is the functional split, especially Split 7.2, which separates lower-PHY processing at the Radio Unit (O-RU) from the more computationally intensive upper-PHY operations at the Distributed Unit (O-DU) [3, 1, 2].

This shift also moves RAN execution away from fixed-function telecommunications equipment toward a cloud-native operating model [3, 1, 2]. In virtualized RAN (vRAN) deployments, network functions are decoupled from specific hardware and run as containerized microservices on Commercial Off-The-Shelf (COTS) servers that collectively form the O-Cloud [3, 1]. The Service Management and Orchestration (SMO) framework then applies standard cloud-orchestration principles to lifecycle management, scaling, and deployment across that substrate, while supporting multi-vendor interoperability and programmatic control [1, 2]. AI/ML-driven automation extends this model further by coordinating both computational and communication resources [2, 1].

However, realizing this cloud-native flexibility introduces severe execution challenges, primarily due to the stringent, deterministic nature of cellular telecommunications. RAN functions demand substantial compute and network resources. The network operates in a strict time-slotted fashion, with slot durations ranging from hundreds of microseconds to just one millisecond [8]. During a single slot, the system must execute radio resource allocation in the control plane and process user packets in the data plane (e.g., segmentation, header attachment, and retransmissions) [8].

Executing these slot-level tasks on standard cloud infrastructure is demanding. CHRONOS shows that VM interruption and network latency can consume a large share of the slot budget, so faithful cloud execution depends on tight release synchrony and careful control of timing jitter [8]. This timing pressure helps explain why recent O-RAN and AI-RAN work places disaggregated DU/CU software, O-Cloud resources, and accelerator support within a common cloud-native operating model, rather than treating real-time RAN functions as fixed-purpose appliances [3, 1, 2].

2.1.2. The AI-RAN Paradigm

As cellular systems move toward 6G, the RAN is increasingly framed as both a communications system and a distributed compute platform. The AI-RAN literature describes this convergence as a joint evolution of infrastructure, control, and services, with AI capabilities embedded into the RAN stack and coordinated across edge and cloud resources [1, 2, 3].

Recent literature commonly organizes this convergence into three closely related pillars [2, 3]:

- **AI-for-RAN (Network Optimization):** This pillar applies AI to improve internal network functions, including scheduling, beam management, traffic steering, energy saving, and anomaly handling [2, 3]. Here, AI remains subordinate to the primary telecommunications service.
- **AI-on-RAN (Native AI Services):** This pillar treats the RAN and its edge compute continuum as a native platform for AI services, especially distributed learning and low-latency inference placed close to DU/CU resources [2].
- **AI-and-RAN (Infrastructure Multiplexing):** This pillar asks when spare O-Cloud or AI-RAN compute can host secondary AI workloads without breaking primary radio deadlines [2, 6, 1].

This research proposal focuses strictly on the **AI-and-RAN** pillar. Relative to the broader AI-for-RAN literature, it remains at an earlier stage of system design and validation, even though it offers a clear economic reason to reuse AI-RAN infrastructure beyond peak radio demand periods [2, 6, 1].

In practice, that reuse depends on turning spare DU or O-Cloud compute into a controlled edge-service substrate while preserving the primacy of RAN timing and reliability requirements [1, 6, 8]. The central systems question is therefore not whether AI workloads can be injected in the abstract, but how to admit and coordinate only those workloads that remain compatible with underlying radio deadlines [2, 6].

2.1.3. The Anatomy of Latency-Critical Inference Workloads

Latency-sensitive edge AI workloads are also part of this background. CORA models their end-to-end service time as a joint uplink, compute, and downlink problem, while recent device,

RAN, and cloud measurements show that strict edge feasibility depends on workload variant and placement rather than on accelerator presence alone [4, 5].

As a result, edge inference is a multi-resource scheduling problem whose bottleneck can shift with channel quality, traffic load, and input/output size [4]. For generative services, this difficulty is amplified by the fact that different stages of inference place different demands on the platform [5].

The first stage is the **Prefill Phase** (or prompt processing). In this stage, the model processes the prompt in parallel and establishes the time-to-first-token budget. This phase is typically more compute-heavy than later generation, which is why TTFT becomes a useful operational signal under strict edge SLAs [5].

The second stage is the **Autoregressive Decoding Phase**. Modern serving systems consistently distinguish it from prefill: prefill is prompt-parallel and typically compute-dense, so it dominates time-to-first-token, whereas decoding is token-serial and advances one token at a time [5, 9, 10]. Because each decode step must reuse the accumulated KV cache while doing comparatively little arithmetic, its bottleneck is usually memory movement and KV-cache management rather than raw ALU saturation [11, 9]. This is why steady per-token latency or throughput behaves differently from TTFT under strict edge SLAs [5].

The same phase split is visible in the design of current serving engines. PagedAttention/vLLM shows that the KV cache is large, dynamically sized, and easily wasted by fragmentation or duplication, so explicit KV-cache memory management directly affects feasible batch size and serving throughput [11]. Sarathi-Serve shows that although batching decodes is throughput-efficient, naive interleaving of prefills and decodes creates interference; its chunked-prefill scheduler is designed to admit new requests without stalling ongoing decodes [9]. DistServe goes further and shows that under tight TTFT and per-token latency constraints, colocating both phases can couple their resource-allocation plans badly enough that some systems explicitly disaggregate prefill and decoding onto different GPUs [10].

For AI-RAN, this means GPU “spare capacity” cannot be inferred from utilization counters alone. During decoding, low SM or ALU utilization can coexist with high pressure on HBM, caches, and KV-state traffic, so a scheduler that interprets low compute utilization as safe headroom can admit RAN work precisely when contention risk remains high [11, 9, 10]. CORA and recent device, RAN, and cloud evidence reinforce that this risk is end-to-end: feasibility depends on model variant, placement, channel quality, and queueing across the uplink, compute, and downlink path, not on accelerator presence alone [4, 5].

2.1.4. Strict RAN Service Level Agreements (SLAs) vs. IT Workloads

Sharing this cloud-native hardware introduces a real clash of scheduling objectives. In standard IT clouds, schedulers can chase fairness or average utilization. In the RAN, CHRONOS shows that VM interruption and network latency can already consume a large fraction of the slot budget, so release synchrony and timing jitter are first-order constraints rather than implementation details [8]. Any coexistence policy for DU-adjacent GPUs therefore has to preserve a strict “RAN-first” priority model instead of a best-effort cloud policy [8, 6].

2.1.5. The Hardware Dichotomy: Training vs. Inference

Within the AI domain, the workload type matters as much as the accelerator itself. Weaver shows why foundation-model training is the friendlier co-located case: LDPC-heavy RAN processing is largely bandwidth-dominated, while training is more compute-heavy and elastic, so spare

SM envelopes can be harvested for secondary work without assuming the same resource profile as the radio path [6]. By contrast, CORA and recent device, RAN, and cloud measurements frame inference as an interactive end-to-end service whose feasibility depends on model variant, placement, channel quality, and deadline budget rather than on GPU presence alone [4, 5]. A scheduler that works for elastic training therefore cannot be carried over directly to latency-critical inference [6, 4, 5].

2.2. Related Work

2.2.1. Hardware-Level GPU Sharing Mechanisms

To realize AI-and-RAN, the GPU must be shared between the RAN DU and the inference stack, but the available mechanism choices mostly expose a trade-off rather than a complete answer. Spatial partitioning such as MIG offers strong isolation and is therefore a useful baseline when strict SLA classes need dedicated slices, yet static partitions are a poor fit for bursty spare compute and can strand capacity that arrives at slot timescales [6, 5]. More elastic temporal sharing can expose more headroom, but Weaver treats mechanism overhead and sharing jitter as central concerns for RAN-safe reuse [6]. These primitives matter, but they do not by themselves provide an inference-aware micro-scheduling policy for a shared DU GPU [6, 5].

2.2.2. Edge Multi-Tenancy and Orchestration

Providing AI inference as a multi-tenant service also requires a slow-timescale orchestration layer. Paradrop is a useful precedent here because it shows that lightweight, containerized services can be deployed at the network’s extreme edge with backend orchestration and explicit resource controls [7]. That said, Paradrop targets lightweight edge services on constrained nodes, while AI-RAN proposals extend orchestration toward the O-Cloud and SMO stack for joint RAN and AI resource management [7, 1]. The orchestration lesson is valuable, but it still has to be paired with a fast local execution policy for shared accelerators.

2.2.3. End-to-End Latency Coordination

Inference latency spans uplink transport, edge execution, and downlink return, and CORA is the strongest direct template in this corpus for coordinating those stages [4]. CORA explicitly budgets latency across uplink, compute, and downlink, derives RB demand from I/O size and channel quality, derives SM demand from a speedup model, and performs pre-admission costing with lightweight planner overhead of about 0.18 ms and roughly 112 B of metadata per request [4]. This is strong evidence that end-to-end latency budgeting is feasible, but CORA still treats compute as a schedulable stage budget rather than as a shared RAN-GPU micro-scheduling problem [4, 5].

2.2.4. RAN-Aware Co-scheduling

Weaver comes closest to the local sharing problem because it embeds a RAN-first spare-compute controller into the MAC scheduler and exports slot-bounded GPU headroom for secondary AI work [6]. Its profiling is concrete: even a pessimistic four-DU peak-load setup still leaves about 40% spare SM capacity, and production-trace replays show spare capacity can remain around 85% during peak daytime periods, although that headroom is bursty within sites and complementary across sites [6]. However, Weaver’s target workload is foundation-model training, not

interactive inference, and the same evidence base warns that LLM decoding is also bandwidth-heavy, which makes it a weaker sharing match with LDPC than training is [6, 5].

2.3. Summary of Research Gap

Taken together, the literature covers the surrounding layers but not the core inference-sharing problem. CHRONOS explains why slot-realistic timing leaves little slack for interference, while Paradrop supplies a pattern for multi-tenant edge orchestration [8, 7]. CORA supplies end-to-end latency budgeting, and Weaver shows how compatible secondary workloads can harvest spare RAN GPU capacity [4, 6]. What is still missing is an inference-specific micro-scheduler for shared RAN GPUs, one that reasons about phase-sensitive inference behavior and admits only those requests whose latency and resource profile remain compatible with slot-level DU deadlines [6, 4, 5].

3. Methodology

To test the core hypothesis and develop the proposed multi-tenant AI-RAN architecture, this research will adopt a phased, systems-engineering methodology.

Phase 1: Workload Profiling and Contention Analysis.

Before a scheduler can be designed, the exact resource demands of both workloads must be established. We will profile standard 5G DU baseband processing alongside a suite of edge AI models (LLMs/VLMs) to establish their precise compute-bound versus memory-bound characteristics. Co-locating these workloads will identify exactly which hardware resources (L2 Cache, memory bandwidth, PCIe bus) cause RAN SLA violations.

Phase 2: Micro-Level Scheduler Design.

Based on profiling data, a dynamic MAC-layer scheduler will be developed. Instead of relying on static partitioning, it will adapt NVIDIA Green Contexts and sub-millisecond preemption protocols to safely inject memory-bound inference tasks into RAN idle slots without compromising the primary telecommunication deadline.

Phase 3: Macro-Level Orchestration Integration.

The scheduler will be integrated into a broader cloud-native framework to achieve multi-tenancy. This involves extending standard orchestration principles (such as Kubernetes pods or Paradrop-style isolated environments) to handle tenant isolation and implement the decoupled end-to-end latency planning mechanism.

4. Project Plan and Timeline

This section does not count towards the page limit.

Milestone	Week	Description
M_1	3	Completion of baseline hardware profiling and bottleneck identification
M_2	6	Micro-scheduler logic implementation and initial testing
M_3	9	Full system evaluation and data collection completed
M_4	12	Submission of final dissertation

Table 1: Project milestones.

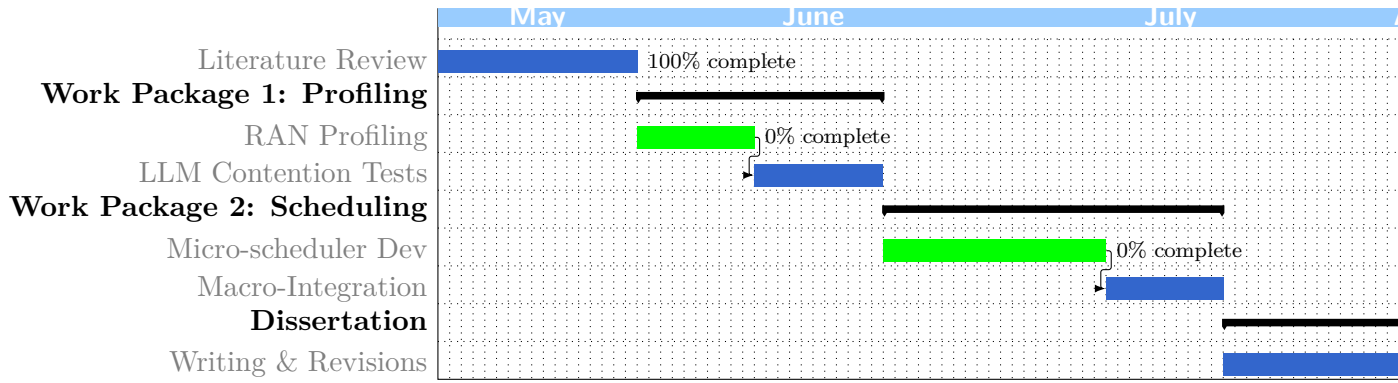


Figure 1: Gantt Chart of anticipated MSc dissertation activities.

Deliverable	Week	Description
D_1	4	Interim report on hardware contention limits
D_2	8	Codebase: Simulated hierarchical scheduling environment
D_3	12	Final Dissertation

Table 2: Project deliverables.

References

- [1] Michele Polese, Niloofar Mohamadi, Salvatore D’Oro, Leonardo Bonati, and Tommaso Melodia. Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G. *arXiv preprint arXiv:2507.06911*, 2025.
- [2] Chenyuan Feng, Howard H. Yang, Kun Guo, Wenchao Xia, Chenxi Liu, and Tony Q. S. Quek. AI-RAN: The pathway to future wireless networks. *Journal of Information and Intelligence*, 4(1):5–22, 2026.
- [3] *Proceedings of the 2025 2nd ACM Workshop on Open and AI RAN*, Hong Kong, China, 2025. Association for Computing Machinery. Part of MobiCom ’25.
- [4] Sunghyun Jin, Serae Kim, Sangtae Ha, and Kyunghan Lee. End-to-End Coordination of RAN and Edge Server for Latency-Critical Inference Serving over Cellular Networks. *Proceedings of the ACM on Networking*, 3(CoNEXT4):1–23, 2025. Article 40.
- [5] Hariz Yet, Nguyen Thanh Tam, Mao V. Ngo, Lim Yi Shen, Lin Wei, Jihong Park, Binbin Chen, and Tony Q. S. Quek. SLA-Aware Distributed LLM Inference Across Device-RAN-Cloud. In *IEEE INFOCOM Workshops 2026 (6G AI-RAN 2026)*, 2026. Preprint on arXiv:2602.23722.
- [6] Weaver: Foundation Model Training over AI-RAN Compute Infrastructure, 2026. Local ACM-formatted manuscript dated March 2026.
- [7] Peng Liu, Dale Willis, and Suman Banerjee. Paradrop: Enabling Lightweight Multi-tenancy at the Network’s Extreme Edge. In *IEEE/ACM Symposium on Edge Computing (SEC)*, pages 1–13, 2016.
- [8] Ujjwal Pawar, Andrew E. Ferguson, Yuto Takano, Jon Larrea, Xenofon Foukas, Mahesh K. Marina, and Bozidar Radunovic. Towards Scalable and Cost-Effective RAN Emulation Leveraging the Public Cloud. In *Proceedings of the 26th International Workshop on Mobile Computing Systems and Applications*, HotMobile ’25, pages 43–48, 2025.
- [9] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gula-vani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. *arXiv preprint arXiv:2403.02310*, 2024.
- [10] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language

- Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, Santa Clara, CA, 2024. USENIX Association.
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv preprint arXiv:2309.06180*, 2023. SOSP 2023.

This section does not count towards the page limit. References should include embedded DOI links.